

SOFTWARE REQUIREMENT SPECIFICATION

QoE Mobile Network Monitoring Application

Table of Contents

1. Introduction
2. Overall Description
3. System Architecture
4. Functional Requirements
5. Non-Functional Requirements
6. External Interface Requirements
7. Data Management Requirements
8. Security and Privacy Requirements
9. System Constraints
10. Future Enhancements
11. Conclusion

1. Introduction

1.1 Purpose of the System

The purpose of this system is to develop a native Android mobile application capable of collecting both subjective Quality of Experience (QoE) feedback and objective network performance indicators from mobile subscribers in Cameroon.

The system is designed to bridge the gap between traditional network-side Quality of Service (QoS) monitoring and actual user-perceived mobile network experience.

Existing telecom monitoring systems provide operators with technical infrastructure data but do not adequately capture real-time user experience, contextual network issues, and subscriber perception. This system addresses that limitation by combining automated background network monitoring with event-triggered user feedback collection.

1.2 Problem Statement

Mobile network users in Cameroon heavily rely on mobile internet for communication, education, work, streaming, and social interaction. However, users continue to experience poor network quality including:

- Slow internet speed
- Signal instability
- Network outages
- High latency
- Video buffering
- Poor call quality

Despite telecom providers possessing strong QoS monitoring systems, there remains limited visibility into actual subscriber Quality of Experience.

The proposed system aims to provide a lightweight, privacy-conscious, and scalable QoE monitoring platform capable of collecting contextual user-centered network experience data.

1.3 Scope of the System

The system focuses on Android smartphones and supports:

- Background network monitoring
- Subjective user feedback collection
- Signal strength monitoring
- Network type detection
- Latency measurement
- Offline-first data storage
- Cloud synchronization
- User QoE dashboard
- Aggregated operator reports

The system does not attempt to access privileged telecom infrastructure metrics or operator internal QoS systems.

1.4 Target Users

The system targets three categories of users:

Subscribers

Mobile users who install the application to monitor their personal network experience and contribute QoE data.

Network Operators

Telecom operators such as MTN and Orange who use anonymized aggregated reports to understand subscriber experience patterns.

System Administrators

Developers and backend administrators responsible for maintaining the infrastructure and ensuring data integrity.

2. Overall Description

2.1 Product Perspective

The application is a standalone native Android system that continuously collects lightweight contextual network indicators and user experience feedback.

The system follows an Offline-First architecture where all collected data is stored locally first before synchronization with the backend cloud infrastructure.

The application combines:

- Passive objective monitoring
- Active subjective feedback
- Local analytics
- Cloud-based aggregation

2.2 Product Functions

The system shall:

- Collect subjective QoE ratings from users
- Collect objective network indicators from the device
- Detect network degradation events
- Trigger contextual feedback prompts
- Store collected data locally
- Synchronize data to cloud servers
- Display personal QoE history to users
- Generate anonymized operator analytics
- Operate efficiently with minimal battery and data usage

2.3 Operating Environment

The application shall operate on:

Component	Specification
Platform	Android
Minimum Android Version	Android 8.0 (Oreo)
Programming Language	Kotlin
Backend	Supabase
Network Connectivity	Mobile Data / WiFi
Local Database	Room Database

2.4 Design Constraints

The system is constrained by:

- Android background execution restrictions
- Permission limitations on newer Android versions
- Battery optimization policies
- User privacy concerns
- Device manufacturer differences
- Internet connectivity availability

2.5 Assumptions and Dependencies

The system assumes:

- Users grant required permissions
- Users possess Android smartphones
- Internet connectivity becomes periodically available for synchronization
- Users allow notification access
- Backend services remain operational

3. System Architecture

3.1 Architectural Pattern

The application adopts the MVVM (Model-View-ViewModel) architecture combined with Clean Architecture principles.

This architecture was selected because it provides:

- Separation of concerns
- Lifecycle awareness
- Scalability
- Improved maintainability
- Better background task management
- Easier testing and debugging

3.2 Architectural Layers

Presentation Layer

Responsible for:

- User interfaces
- User interaction
- Permission requests
- Dashboard visualization
- Notification handling

Technologies:

- Jetpack Compose
- Android Activities
- ViewModels

Domain Layer

Responsible for:

- Business logic
- QoE analysis
- Event detection
- Feedback trigger logic
- Network quality evaluation

Data Layer

Responsible for:

- Local database access
- API communication
- Data synchronization
- Network metric collection
- Repository management

Technologies:

- Room Database
- Retrofit
- Supabase APIs

3.3 Monitoring Architecture

The application uses intelligent context-aware monitoring instead of unrestricted continuous monitoring.

Monitoring occurs during:

- Active device usage
- Screen-on periods
- Connectivity changes
- Detected degradation events
- Periodic lightweight background checks

This reduces:

- Battery consumption
- Background restrictions
- User distrust
- Data usage

3.4 Android APIs Used

API	Purpose
TelephonyManager	Signal strength and network type
ConnectivityManager	Connectivity monitoring
WorkManager	Scheduled background tasks
NotificationManager	Feedback prompts
Room Database	Local storage
Retrofit	Backend synchronization
PowerManager	Device activity awareness

4. Functional Requirements

FR1: User Registration and Identification

The system shall generate a unique anonymous device identifier for each installation.

FR2: QoE Feedback Collection

The system shall allow users to submit subjective QoE ratings covering:

- Overall experience
- Internet responsiveness
- Streaming experience
- Call quality
- General satisfaction

FR3: Event-Triggered Feedback Prompts

The system shall trigger feedback prompts when:

- Signal degradation is detected
- Network type changes occur
- Latency spikes are detected
- Connectivity interruptions occur

Users shall also be able to configure feedback frequency.

FR4: Signal Strength Monitoring

The system shall automatically collect:

- Signal strength values
- Signal quality indicators
- Cellular network status

using Android TelephonyManager APIs.

FR5: Network Type Detection

The system shall detect:

- 2G
- 3G
- 4G
- 5G
- WiFi

network connections.

FR6: Latency Monitoring

The system shall periodically measure network latency using lightweight network requests.

FR7: Local Data Storage

The system shall store all collected records locally using Room Database before synchronization.

FR8: Offline Operation

The application shall continue operating even without internet connectivity.

FR9: Cloud Synchronization

The system shall synchronize unsynced records to the Supabase backend when connectivity becomes available.

FR10: User Dashboard

The system shall display:

- Personal QoE history
- Network trend charts
- Signal quality trends
- Usage summaries

FR11: Operator Analytics

The system shall generate anonymized aggregated reports for operators including:

- Average QoE scores
- Time-based degradation patterns
- Network distribution statistics
- Complaint trends

FR12: Data Cleanup

The system shall automatically remove synchronized records older than 7 days from local storage.

5. Non-Functional Requirements

5.1 Performance Requirements

- The application shall maintain low CPU and memory usage.
- Dashboard screens shall load within 3 seconds under normal conditions.
- Background monitoring tasks shall remain lightweight.

5.2 Battery Efficiency Requirements

- Monitoring shall prioritize event-driven collection.
- Continuous aggressive monitoring shall be avoided.
- Synchronization shall default to WiFi-only mode.

5.3 Privacy Requirements

- Personally identifiable information shall not be uploaded.
- Data shall be anonymized before synchronization.
- Users shall have control over participation.

5.4 Security Requirements

- All communication shall use HTTPS.
- Backend access shall use authenticated APIs.
- Local sensitive records shall be encrypted.
- Supabase Row-Level Security shall be enabled.

5.5 Reliability Requirements

- The application shall support offline operation.
- Unsynchronized data shall persist until upload succeeds.
- Background monitoring shall recover after device reboot.

5.6 Scalability Requirements

The backend system shall support increasing user data volume across multiple regions.

5.7 Usability Requirements

- Feedback prompts shall require less than 30 seconds to complete.
- The UI shall remain simple and non-intrusive.
- Dashboard visualizations shall be easy to understand.

6. External Interface Requirements

6.1 User Interface Requirements

The application shall provide:

- Login/setup screens
- Permission request screens
- Dashboard screens
- Feedback forms
- Settings screens
- Notification prompts

6.2 Hardware Interface Requirements

The system interacts with:

- Cellular modem hardware
- Device network interfaces
- Local device storage

through Android system APIs.

6.3 Software Interface Requirements

Component	Interface
Android OS	Android SDK APIs
Backend	REST APIs
Database	Room ORM
Authentication	Firebase Authentication
Analytics	Supabase APIs

6.4 Communication Interfaces

The system shall communicate with backend infrastructure using:

- HTTPS
- JSON payloads
- RESTful APIs

7. Data Management Requirements

7.1 Data Collected

The system shall collect:

- QoE ratings
- Signal strength
- Network type
- Latency measurements
- Device model
- Android version
- Timestamps

7.2 Data Storage Strategy

The application follows an Offline-First architecture:

1. Data collected locally
2. Records marked unsynced
3. Synchronization occurs when internet becomes available
4. Records marked synced after successful upload
5. Old synchronized records purged automatically

7.3 Data Retention

- Unsynced records shall remain until upload succeeds.
- Synced records older than 7 days shall be deleted locally.
- Aggregated backend analytics may remain for long-term analysis.

8. Security and Privacy Requirements

8.1 User Consent

The system shall request user consent before:

- network monitoring,
- notification access,
- data synchronization,
- permission activation.

8.2 Permission Requirements

Permission	Purpose
INTERNET	Network communication
ACCESS_NETWORK_STATE	Connectivity detection
READ_PHONE_STATE	Telephony information
POST_NOTIFICATIONS	Feedback notifications
ACCESS_FINE_LOCATION	Signal and cell information

8.3 Privacy Protection

The system shall:

- avoid collecting personal identifiers,
- anonymize uploaded records,
- provide transparency to users,
- allow users to stop monitoring at any time.

9. System Constraints

The system is constrained by:

- Android battery optimization policies
- OEM-specific process restrictions
- Permission sensitivity on modern Android versions
- User privacy concerns
- Internet availability
- Device hardware differences

The system cannot access:

- telecom core network data,
- privileged carrier APIs,
- operator infrastructure systems.

10. Future Enhancements

Future versions may include:

- AI-based QoE prediction
- Heatmap visualization
- Packet loss monitoring
- Jitter analysis
- Cross-platform support
- Advanced operator analytics
- Predictive congestion detection

11. Conclusion

The proposed QoE mobile monitoring system provides a technically feasible solution for collecting real-time user-centered network experience data in Cameroon.

The system combines subjective QoE feedback with objective device-level network indicators using a lightweight Android-native architecture.

The SRS defines a scalable, privacy-conscious, and maintainable system aligned with Android platform capabilities, stakeholder expectations, and real-world mobile monitoring constraints.

The adoption of MVVM with Clean Architecture principles, Offline-First synchronization, event-driven monitoring, and anonymized analytics provides a strong technical foundation for subsequent system design and implementation phases.